PurifyFL: Non-Interactive Privacy-Preserving Federated Learning Against Poisoning Attacks Based on Single Server

Yanli Ren, Zhe Yang, Guorui Feng, and Xinpeng Zhang

***Abstract*—Privacy-preserving federated learning (PPFL) allows multiple users to collaboratively train models on local devices without the the risk of privacy leakage. However, PPFL is prone to be disrupted by poisoning attacks for the server being forbbiden from accessing users' updates. The existing protocols focusing on poisoning attacks in PPFL generally use two servers to interactively execute protocols to defend against poisoning attacks, while the other ones using a single server require multiple rounds of server-user interactions, both of which incur significant communication overheads. We propose PurifyFL, a privacy-preserving poisoning attacks defense strategy. PurifyFL only relies on a single server while most of the previous works depend on two non-colluding servers, which are impractical in reality. Moreover, We also achieve non-interactivity between the users and the server. Experiments show that PurifyFL can effectively resist typical poisoning attacks with lower computational and communication overheads compared to existing works.**

***Index Terms*—Federated learning, privacy-preserving, homomorphic encryption, poisoning attacks.**

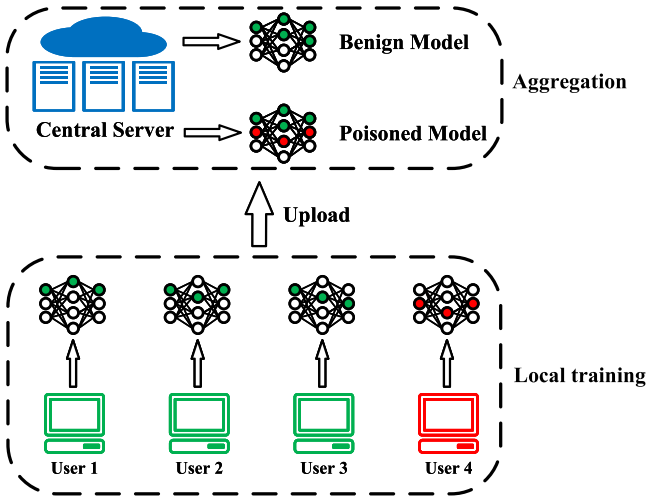


Fig. 1. Poisoning attacks in federated learning. Malicious users launch poisoning attacks by uploading poisonous models to disrupt federated learning.

I. Introduction

ARTIFICIAL intelligence has achieved widespread success in fields such as natural language processing and image recognition. However, traditional AI training requires vast amounts of data, which cannot be collected in some sensitive areas. Diego et al. show that structural properties of political corruption, police intelligence, and money laundering networks can be used to recover missing criminal partnerships, distinguish among different types of criminal and legal associations with outstanding accuracy [1]. Further, Haroldo et al. explore the potential of graph convolutional networks to learn patterns among networked criminals, and develop a series of deep learning models based on the GraphSAGE framework which can produce high-quality embeddings for node and edge properties [2]. However, given the sensitivity of crime data, especially in the context of transnational crime, collecting such data is undoubtedly challenging. Federated learning as a burgeoning collaborative machine learning approach is effective in overcoming the challenge of data islands in artificial intelligence [3]. Since no participant has access to anyone else's data, FL is suitable for scenarios where there are legal concerns about sensitive data such as medical data, data from personal devices, and other data containing users' privacy that cannot be aggregated [4].

Unfortunately, FL faces many security issues such as privacy leakage and poisoning attacks [5]. Although FL prevents servers from directly accessing users' local data, analysis of the existing works shows that users' local updates can reveal privacy. Melis et al. [6] designs active and passive attack methods where poisoners can infer other participants' data by the updates. The works [7], [8] show that the server can reconstruct users' train data by the updates. Hence, it is imperative to protect the updates to prevent privacy leakage. In PPFL, some of the users participating in the training may be malicious. They can potentially upload poisonous updates to interfere with the aggregation process, manipulating the model and compromising its accuracy. As shown in Fig. 1, user 4 is a poisoner whose uploaded updates are poisonous. The target of FL is to aggregate a high-quality aggregated model, but the server may generate a low-quality model because of the interference of poisonous updates. Due to

Received 19 June 2024; revised 31 October 2024; accepted 25 December 2024. Date of publication 17 March 2025; date of current version 29 May 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62476164 and in part by the Natural Science Foundation of Shanghai under Grant 22ZR1481000. *(Corresponding author: Yanli Ren.)*

The authors are with the School of Communication and Information Engineering, Shanghai University, Shanghai 200444, China (e-mail: renyanli@shu.edu.cn; yangzheshanghai@shu.edu.cn; grfeng@shu.edu.cn; xzhang@shu.edu.cn).

Recommended for acceptance by Y. Lu.
Digital Object Identifier 10.1109/TETCI.2025.3540420

2471-285X © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: National University of Modern Languages. Downloaded on July 31,2025 at 07:01:33 UTC from IEEE Xplore. Restrictions apply.

the strict protection of users' privacy in PPFL, poisoning attacks become more covert.

II. Realted Works

In terms of preventing privacy leakage, the works [9], [10], [11] introduce differential privacy (DP) into FL to avoid privacy leakage. DP preserves sensitive information by injecting fuzzy noise into local updates and has advantages in computational efficiency and communication costs. However, the noise-induced degradation for the accuracy of the model is a critical shortcoming of DP-based FL. Besides DP, secure multi-party computation (MPC) is also applied in FL. MPC-based FL [12], [13], [14] generally preserves privacy by adding masks to the updates, and the effect of masks is canceled during the aggregation. However, in MPC-based FL, the system spends a significant amount of additional communication overhead to handle dropouts to ensure the correctness of the aggregation results. Homomorphic encryption (HE) allows operations on ciphertexts, which has been widely employed in FL. HE-based FL [15], [16], [17] can securely aggregate the updates of users. Compared with DP and MPC, HE-based FL neither loses the accuracy of the model nor has to handle dropout users.

To mitigate the negative impact of poisoning attacks, several similarity-based and statistics-based defense strategies have been proposed [18], [19], [20], [21]. The statistics-based method selects the updates of benign users according to statistical properties such as median and mean, and estimates the global update accordingly. In [18], both median-based and trimmed-mean-based rules are proposed. The median strategy counts the median of each dimension of the updates to synthesize a median update and takes it as the global update. In the trimmed-mean rule, for each dimension, the server sorts the updates from all users and discards extreme values, then takes the mean of the remaining values as the global update. The similarity-based approach adopts metrics such as Euclidean distance and cosine similarity to evaluate the reliability of each user's uploaded update and aggregate them based on similarity, with representative protocols such as Krum [19], Bulyan [20] and FL-Trust [21]. For each user's update, Krum calculates its Euclidean distance from the rest of the users' updates and the update with the smallest sum of Euclidean distance from the other updates is selected as the global update. Bulyan improves Krum by trimmed-mean, which first selects multiple updates by Krum, and then aggregates these updates. FL-Trust assumes that the server holds a validation dataset, and determines whether a user's update is malicious based on the updates trained from the validation dataset. Yazdinejad et al. introduce an internal auditor that evaluates encrypted gradient similarity and distribution to differentiate between benign and malicious gradients, employing a Gaussian Mixture Model and Mahalanobis Distance for byzantine-tolerant aggregation to improve the accuracy of the model [22].

The aforementioned protocols do not take privacy preservation and poisoning attacks into consideration at the same time and all of them suffer from security risks. Liu et al. and Ma et al. design a series of protocols that preserve privacy and mitigate poisoning attacks by utilizing homomorphic encryption [23], [24]. In these protocols, two non-colluded servers perform secure computation protocols interactively to calculate the similarities between encrypted updates and identify potential poisonous updates based on the similarity. Zhang et al. design a single-server-based protocol DetectPMFL in order to detect unreliable users in PPFL [25], which share some similarities by detecting poisoning attacks. Differing from [23], [24], DetectPMFL introduces additional interactions between users and the server. Recently, Chen et al. review the state-of-the-art privacy-preserving federated learning protocols, and evaluate all discussed protocols in terms of efficiency and security properties [26]. Currently, there is no PPFL protocol that can defend against poisoning attacks based on a single server without extra interactions.

In response to these challenges, we propose PurifyFL, which can also preserve users' privacy and defend against poisoning attacks based on a single server, and there is no extra interaction between the server and users. We summarize the contributions of our work as follows.

- We propose PurifyFL, a PPFL protocol based on homomorphic encryption, which can protect the local updates and the aggregated update simultaneously. Adversaries can not infer users' updates.
- PurifyFL identifies the potential poisonous updates based on magnitude and direction, and can effectively mitigate the typical poisoning attacks. It also achieves verifiability. The server can verify the correctness of users' uploaded information, which ensures the correctness of aggregated results.
- PurifyFL only relies on a single server while the previous ones require two non-colluding servers, which are impractical in reality. The cryptosystem we employed supports both addition and multiplication on ciphertexts, thereby eliminating the need for two servers.
- PurifyFL achieves non-interactivity by splitting the private key of the system. In each epoch, there is no additional interaction between users and the server except for uploading and downloading.

III. Preliminaries

In this section, we briefly introduce homomorphic encryption, federated learning and poisoning attacks.

A. CKKS Homomorphic Encryption

The CKKS (Cheon, Kim, Kim and Song) cryptosystem [27] is a level homomorphic encryption scheme that supports addition and multiplication on ciphertexts, including initialization, encryption, and decryption algorithms. The security of the CKKS cryptosystem is based on the ring learning with error (RLWE) problem.

Initialization: Given integer k, q_0, p, P and L, compute $N = 2^k$, $Q = q_0 \cdot p^L$. Let $Z[X]/(X^N + 1)$ be the ring R and $Z[X]_Q/(X^N + 1)$ be R_Q. Let x_s, x_e and x_r be distributions over R. Choose $s \leftarrow x_s$, $a \leftarrow R_Q$ and $e \leftarrow x_e$, and compute $b = -s \cdot a + e mod Q$. The private and public keys are $(1, s)$

Authorized licensed use limited to: National University of Modern Languages. Downloaded on July 31,2025 at 07:01:33 UTC from IEEE Xplore. Restrictions apply.

and (b, a), respectively. Choose $a' \leftarrow R_{PQ}$ and e', and compute $b' = -s \cdot a' + e' + P \cdot s^2 modP \cdot Q$. The evaluation key is $evk = (b', a')$.

$Encrypt(pk, m)$: The CKKS cryptosystem supports single instruction multiple data (SIMD) operation, which allows users to encrypt at most $N/2$ messages at a time. Given plaintext $m = (m_1, m_2, \ldots, m_{N/2})$, m is encoded as a polynomial $m(x) = \lfloor \Delta \cdot IFFT(m) \rceil$, where Δ is a big integer and $IFFT$ denotes inverse fast Fourier transform. Choose $r \leftarrow x_r$ and $e_0, e_1 \leftarrow x_e$, and output ciphertext $[\![m]\!] = r \cdot pk + (m(x) + e_0, e_1) modQ = (c_0, c_1)$.

$Decrypt(sk, [\![m]\!])$: Given a ciphertext $[\![m]\!]$, $[\![m]\!]$ is decrypted to a polynomial by computing $[\![m]\!] \cdot sk = c_0 + c_1 \cdot s = e \cdot r + m(x) + e_0 + e_1 \cdot s modQ \approx m(x)$. Then $m(x)$ is decoded to the original message m by computing $FFT(m(x)/\Delta) \approx m$, where FFT denotes fast Fourier transform. For simplicity, the message polynomial $m(x)$ is equivalent to the original message m in this paper.

Given plaintexts m, m' and the corresponding ciphertexts $[\![m]\!] = (c_0, c_1)$ and $[\![m']\!] = (c_0', c_1')$, the homomorphic addition is as follows: $([\![m]\!] + [\![m']\!]) \cdot s \approx m + m'$. The homomorphic multiplication is as follows: $(d_0, d_1, d_2) = (c_0 c_0', c_0 c_1' + c_1 c_0', c_1 c_1') modQ$, $c_{mult} = (d_0, d_1) + \lfloor P^{-1} d_2 \cdot evk \rceil$, $c_{mult} \cdot s \approx m \cdot m'$.

B. Federated Learning

The goal of federated learning is to train the model on a large amount of decentralized data [28]. Suppose that there are n users participating in FL. User U_i $(i \in [1, n])$ holds a private dataset D_i. At the t-th epoch, all the users independently train global model ω^{t-1} locally. After local training, U_i obtains the local model of t-th epoch ω_i^t. Then U_i calculates the local update θ_i^t:

$$\theta_i^t = \omega_i^t - \omega^{t-1}, \tag{1}$$

and uploads it to the server.

The server aggregates the local updates of users to generate a global update θ^t such that

$$\theta^t = \frac{1}{n} \sum_{i=1}^{n} \theta_i^t. \tag{2}$$

Then the server returns it to U_i. U_i updates the local model as follows:

$$\omega^t = \omega^{t-1} + \theta^t. \tag{3}$$

The aforementioned process will be iteratively executed until reaching the preset number of epochs or the model converges. For ease of handling, we concatenate the model parameters into one-dimensional vectors of floating-point numbers in this paper.

C. Poisoning Attacks

Poisoning attacks in federated learning can be divided into targeted attacks and untargeted poisoning attacks.

- Targeted poisoning attacks: The deployment of targeted poisoning attacks relies on the modification of training data, such as mislabelling the data (label-flipping attack) [29] and adding triggers to the data (backdoor attack) [30]. The model trained on modified data will cause the global model to output the poisoners' pre-set labels for certain inputs instead of the real labels.
- Untargeted attacks: Untargeted attacks aim to reduce the overall accuracy of the model rather than a specific class [31]. Attackers can launch untargeted attacks by shuffling the labels of training data, degrading the quality of training data with Gaussian noise, or directly uploading randomly generated models, among other methods.

Both of the above attacks are implemented by uploading poisonous updates. Therefore the key to defend against poisoning attacks lies in identifying the poisonous updates and keeping them out of aggregation.

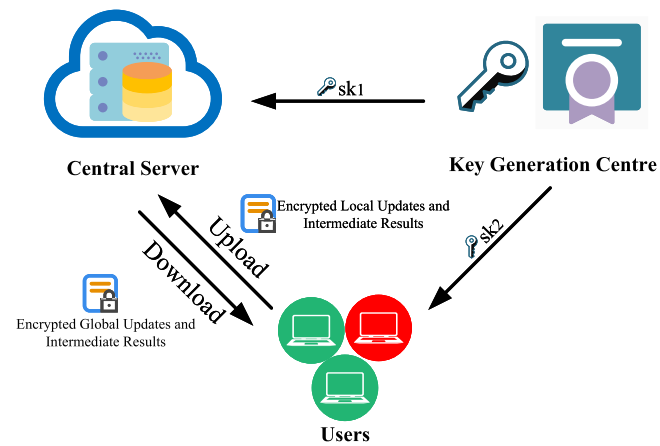


Fig. 2. The system model. The system contains three entities: a trusted authority, users, and a server. The trusted authority generates encryption parameters for the server and users. Users train models on local datasets collaboratively. The server collects users' models and aggregates them.

IV. Problem Statement

We formalize the system model and threat model of the proposed PurifyFL in this section.

A. System Model

As shown in Fig. 2, our system model adopts the classic architecture of PPFL, including the trusted authority, the server responsible for aggregation and users.

- *Trusted authority (TA):* The TA is responsible for generating and broadcasting public parameters for encryption. TA randomly splits the private key sk into sk_1 and sk_2 $(sk = sk_1 + sk_2)$ and distributes them to the server and users.
- *Central server (CS):* The CS undertakes the tasks of collecting and aggregating users' encrypted updates while defending against poisoning attacks.
- *Users:* Users train the global model locally, including both semi-honest users and poisoners. Regardless of whether users are semi-honest or malicious, the updates they upload are encrypted, preventing the server from accessing the plaintexts.

Authorized licensed use limited to: National University of Modern Languages. Downloaded on July 31,2025 at 07:01:33 UTC from IEEE Xplore. Restrictions apply.

B. Threat Model

We consider four potential adversaries who could compromise users' privacy: CS, poisoners, semi-honest users, and eavesdroppers in the channel.

- *CS:* The CS is semi-honest, which faithfully executes the pre-set protocol, while also attempting to learn the updates uploaded by users.
- *Semi-honest users:* Semi-honest users faithfully execute the pre-set protocol and try to analyze the private information of other users.
- *Poisoners:* Poisoners launch poisoning attacks to disrupt federated learning. We assume that poisoners can eavesdrop on messages transmitted in the channel, meaning they can access any message sent by users to the server. Additionally, for purposes such as enhancing poisoning attacks and inferring users' updates, poisoners will collude with each other. Prior works [23], [24] assume that the proportion of poisoners is less than 50%, and we follow this setting.
- *Eavesdroppers:* In this paper, all the communications are executed in the public channel. Hence, we assume that there exist attackers who can eavesdrop on all the information transmitted in the channel to obtain users' local updates and the global update.

Note that poisoners will not collude with the CS. Poisoners aim to manipulate FL by submitting encrypted poisonous updates, whereas if they collude with the server by leaking sk_2, it will cause the leakage of their privacy, to no benefit of their own. Therefore, we consider that the poisoners will not choose to collude with the CS, and prior works [23], [24] have all employed such a setting.

V. Proposed PurifyFL Protocol

In this section, we first give an overview of PurifyFL and then define some functions that are used in our protocol. Finally, we introduce the proposed PurifyFL in detail.

A. Overview

An update can be thought of as a vector with direction and magnitude. In federated learning, the local updates of users reflect the distribution of their local data. This implies that the closer the distributions of users' local data are, the more similar their local updates will be [32]. Poisoners launching poisoning attacks is equivalent to change the distribution of their local data, and therefore their local updates deviate from the majority of semi-honest users inevitably in terms of direction or magnitude. This serves as the theoretical basis for distinguishing poisonous updates from benign ones. We calculate the cosine similarities between all updates and the previous epoch's global update to quantify the difference in their directions. However, the identification of poisoning updates relying solely on direction is insufficient. If a user amplifies his update by a factor of 10 before uploading it, his update would receive a tenfold weight during aggregation, which is not permissible. Therefore, we consider that an update should be excluded from the aggregation if its magnitude significantly exceeds that of the majority of users. Fig. 3 shows our defense strategy.

In terms of privacy protection, we employ homomorphic encryption to guarantee the confidentiality of users' local updates. Previous works [23], [24] employ encryption algorithms that only support homomorphic addition, which incurs a high cost for achieving secure multiplication on ciphertexts as it requires two servers to execute protocols interactively. We employ the CKKS cryptosystem, which provides support for secret multiplication, thereby enabling a single server to compute cosine similarity on encrypted data. However, in the single-server scenario, it is a challenge to decrypt the cosine similarities. The server holding the private key is unacceptable because it implies that the server can also decrypt users' local updates. In [25] the users hold the private key and the server returns the encrypted cosine similarities to users for decryption, which introduces additional interactions between the server and users. PurifyFL solves the above problem by splitting the key, where the server and users separately hold a part of the private key, and the server can decrypt the cosine similarity without disclosing users' updates.

B. Definitions of Functions

Before introducing the details of the proposed protocol, we define several functions that are required for the implementation of the protocol.

$PD([\![m]\!], sk_i)$: Given a ciphertext $[\![m]\!]$ and a partial key sk_i, the pre-decryption of $[\![m]\!]$ is as follows:

$$PD([\![m]\!], sk_i) = [\![m]\!] \cdot sk_i + e \approx [\![m]\!] \cdot sk_i, \tag{4}$$

where e is the noise in x_e.

$CD([\![m]\!] \cdot sk_1, [\![m]\!] \cdot sk_2)$: Given ciphertexts $[\![m]\!] \cdot sk_1$ and $[\![m]\!] \cdot sk_2$, the completed decryption is as follows:

$$\begin{aligned} CD([\![m]\!] \cdot sk_1, [\![m]\!] \cdot sk_2) &= [\![m]\!] \cdot sk_1 + [\![m]\!] \cdot sk_2 \\ &= [\![m]\!] \cdot (sk_1 + sk_2) + e \approx m. \end{aligned} \tag{5}$$

$Sum(\theta_i^t)$: Given a vector $\theta_i^t = (\theta_{i1}^t, \theta_{i2}^t, \ldots, \theta_{im}^t)$, $Sum(\theta_i^t)$ is calculated as follows:

$$Sum(\theta_i^t) = \sum_{k=1}^{m} \theta_{ik}^t \tag{6}$$

$He.Add([\![x]\!], [\![y]\!])$: Given two ciphertexts $[\![x]\!]$ and $[\![y]\!]$, $He.Add$ represents the secret addition:

$$[\![x+y]\!] \leftarrow He.Add([\![x]\!], [\![y]\!]). \tag{7}$$

$He.Mul([\![x]\!], [\![y]\!])$: Given two ciphertexts $[\![x]\!]$ and $[\![y]\!]$, $He.Mul$ represents the secret multiplication:

$$[\![xy]\!] \leftarrow He.Mul([\![x]\!], [\![y]\!]). \tag{8}$$

C. Construction of PurifyFL

1) System Initialization: The TA generates the encryption parameters and allocates the keys. The process of this phase is shown in Algorithm 1.

After generating pk and sk, the TA splits the private key sk into sk_1 and sk_2 and distributes sk_1 to the CS and sk_2 to all users. Then the CS computes $[\![\omega^0]\!]_{sk_1}$ and returns

Authorized licensed use limited to: National University of Modern Languages. Downloaded on July 31,2025 at 07:01:33 UTC from IEEE Xplore. Restrictions apply.

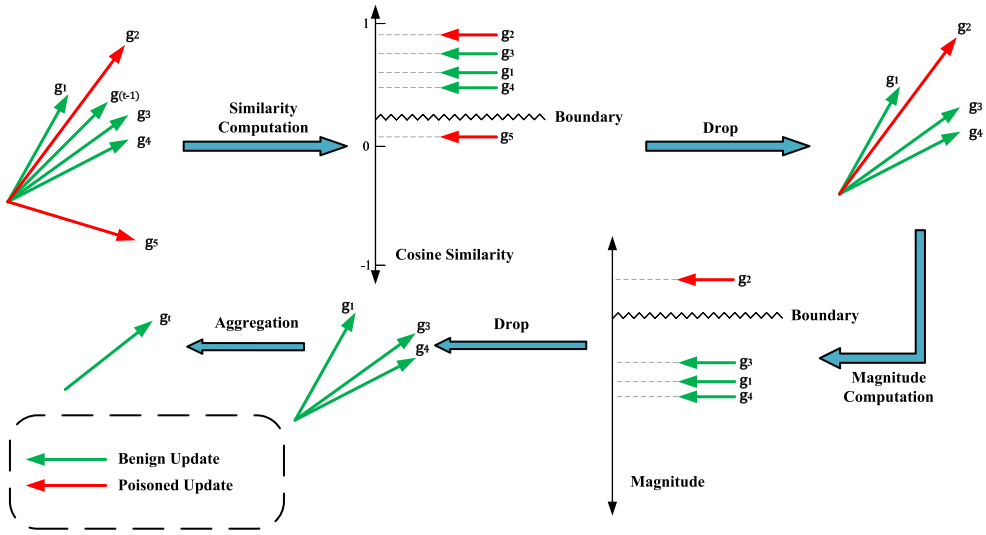


Fig. 3. The proposed defense strategy. The proposed strategy checks model updates from two aspects: direction and magnitude. An update with an abnormal direction or magnitude will be identified as malicious and discarded.

Algorithm 1: System Initialization.

Input: Security parameter τ
Output: Keys pk, evk, sk_1, sk_2

TA:
Run $Initialization$ to generate pk, evk and sk;
Split $sk = (1, s)$ into $sk_1 = (1, s_1)$ and $sk_2 = (0, s_2)$;
return Broadcast pk and distribute sk_1 to the CS and sk_2 to $U_{i\in[1,n]}$;
CS:
$[\![\omega^0]\!] \cdot sk_1 \leftarrow PD([\![\omega^0]\!], sk_1)$;
Distribute $[\![\omega^0]\!]$ and $[\![\omega^0]\!] \cdot sk_1$ to $U_{i\in[1,n]}$;
Users:
for $U_{i\in[1,n]}$ **do**
  $[\![\omega^0]\!] \cdot sk_2 \leftarrow PD([\![\omega^0]\!], sk_2)$;
  $\omega^0 \leftarrow CD([\![\omega^0]\!] \cdot sk_1, [\![\omega^0]\!] \cdot sk_2)$;
end for

$[\![\omega^0]\!]_{sk_1}$ and $[\![\omega^0]\!]$ to all users. Pre-decryption does not compromise the privacy of users, which is proved in Theorem 1. $U_{i\in[1,n]}$ computes $[\![\omega^0]\!]_{sk_2}$ in the same way and decrypts the initial ω^0 by (5).

2) Local Training: Assume that the global model and the global update of $(t-1)$-th epoch are ω^{t-1} and θ^{t-1}, respectively. In the t-th epoch, U_i trains ω^{t-1} on the local dataset. The local update of U_i is denoted as $\theta_i^t = \{\theta_{i1}^t, \theta_{i2}^t, \ldots, \theta_{im}^t\}$, where θ_{im}^t represents the m-th dimension of θ_i^t. In addition to training ω^{t-1} locally, U_i is required to calculate the inner product of θ_i^t and θ^{t-1} on ciphertexts, as well as the magnitude of θ_i^t. The random vectors α and β should satisfy the equation $\alpha_1 + \alpha_2 + \cdots + \alpha_m = \beta_1 + \beta_2 + \cdots + \beta_m$. Algorithm 2 shows the details of this phase.

3) Aggregation: In this phase, the CS defends against poisoning attacks and aggregates benign updates. The process is shown in Algorithm 3 and Algorithm 4.

Algorithm 2: Local Training.

Input: The global update of $(t-1)$-th epoch θ^{t-1} and its ciphertext $[\![\theta^{t-1}]\!]$, local dataset $D_i, i \in [1, n]$.
Output: $[\![\alpha]\!]$, $[\![\beta]\!]$, $[\![\theta_i^t]\!]$, $[\![(\theta_i^t)^2 + \alpha - \beta]\!]_u \cdot sk_2$, $[\![\theta_i^t\theta^{t-1} + \alpha - \beta]\!]_u \cdot sk_2$, $[\![\theta_i^t\theta^{t-1} + (\theta_i^t)^2]\!]_u \cdot sk_2$ and $Sum((\theta^{t-1})^2)$.

In the t-th epoch:
for $U_{i\in[1,n]}$ **do**
  Train ω^t on the dataset D_i to obtain the local update of the $t-$th epoch θ_i^t;
  Generate two random vectors $\alpha = (\alpha_1, \alpha_2, \ldots, \alpha_m)$ and $\beta = (\beta_1, \beta_2, \ldots, \beta_m)$;
  $[\![\alpha]\!] \leftarrow Enc(\alpha, pk)$, $[\![\beta]\!] \leftarrow Enc(\beta, pk)$;
  $[\![\theta_i^t]\!] \leftarrow Enc(\theta_i^t, pk)$;
  $[\![(\theta_i^t)^2]\!] \leftarrow He.Mul([\![\theta_i^t]\!], [\![\theta_i^t]\!])$;
  $[\![(\theta_i^t)^2 + \alpha - \beta]\!]_u \leftarrow He.Add([\![(\theta_i^t)^2]\!], [\![\alpha]\!], -[\![\beta]\!])$;
  $[\![\theta_i^t\theta^{t-1}]\!] \leftarrow He.Mul([\![\theta_i^t]\!], [\![\theta^{t-1}]\!])$;
  $[\![\theta_i^t\theta^{t-1} + \alpha - \beta]\!]_u \leftarrow He.Add([\![\theta_i^t\theta^{t-1}]\!], [\![\alpha]\!], -[\![\beta]\!])$;
  $[\![(\theta_i^t)^2 + \alpha - \beta]\!]_u \cdot sk_2 \leftarrow PD([\![(\theta_i^t)^2 + \alpha - \beta]\!]_u, sk_2)$;
  $[\![\theta_i^t\theta^{t-1} + \alpha - \beta]\!]_u \cdot sk_2 \leftarrow PD([\![\theta_i^t\theta^{t-1} + \alpha - \beta]\!]_u, sk_2)$;
  $[\![\theta_i^t\theta^{t-1} + (\theta_i^t)^2]\!]_u \leftarrow He.Add([\![(\theta_i^t)^2]\!], [\![\theta_i^t\theta^{t-1}]\!])$;
  $[\![\theta_i^t\theta^{t-1} + (\theta_i^t)^2]\!]_u \cdot sk_2 \leftarrow PD([\![\theta_i^t\theta^{t-1} + (\theta_i^t)^2]\!]_u, sk_2)$
  Compute $Sum((\theta^{t-1})^2)$;
end for

The plaintext $(\theta_i^t)^2 + \alpha - \beta$ decrypted by the CS has the following forms:

$$(\theta_i^t)^2 + \alpha - \beta = ((\theta_{i1}^t)^2 + \alpha_1 - \beta_1, \ldots, (\theta_{im}^t)^2 + \alpha_m - \beta_m) \quad (9)$$

Authorized licensed use limited to: National University of Modern Languages. Downloaded on July 31,2025 at 07:01:33 UTC from IEEE Xplore. Restrictions apply.

Since vectors α and β are random, the server cannot determine the value of $(\theta_i^t)^2$, which protects the privacy of U_i. The CS can achieve $Sum((\theta_i^t)^2)$ by computing the following equation:

$$\begin{aligned} Sum((\theta_i^t)^2 + \alpha - \beta) &= Sum((\theta_i^t)^2) + Sum(\alpha) - Sum(\beta) \\ &= Sum((\theta_i^t)^2), \end{aligned} \tag{10}$$

and the calculation of $Sum(\theta_i^t\theta^{t-1})$ is the same. $Sum((\theta_i^t)^2)$ and $Sum(\theta_i^t\theta^{t-1})$ are criteria for determining whether θ_i^t is poisonous, so the malicious user has the incentive to forge them to make the CS believe that θ_i^t is benign. In (10), if the vectors α and β generated by the malicious user do not follow the requirement of $\alpha_1 + \alpha_2 + \cdots + \alpha_m = \beta_1 + \beta_2 + \cdots + \beta_m$, it will cause the CS to compute a wrong $Sum((\theta_i^t)^2)$. Therefore the verifications of $Sum((\theta_i^t)^2)$ and $Sum(\theta_i^t\theta^{t-1})$ are necessary.

If U_i is not a poisoner, it holds that

$$\begin{aligned} &Sum((\theta_i^t)^2 + \alpha - \beta) + Sum(\theta_i^t\theta^{t-1} + \alpha - \beta) \\ &= Sum(\theta_i^t\theta^{t-1} + (\theta_i^t)^2)_s \end{aligned} \tag{11}$$

The CS can achieve verification easily by (11). The verification of $Sum(\theta_i^t\theta^{t-1})$ is similar to $Sum((\theta_i^t)^2)$ and we do not repeat it. If the verification fails, $[\![\theta_i^t]\!]$ will be discarded. Although the CKKS cryptosystem supports rotation operation, allowing the CS to compute $Sum((\theta_i^t)^2)$ and $Sum(\theta_i^t\theta^{t-1})$ on ciphertexts, considering the expensive computational overhead of rotation, we choose to employ random vectors to ensure that the CS can obtain the correct result while protecting the privacy of users.

If the verification is successful, the CS computes the cosine similarity between θ_i^t and θ^{t-1} as follows:

$$Cos_i^t = \frac{<\theta^{t-1}, \theta_i^t>}{\|\theta^{t-1}\|_2\|\theta_i^t\|_2} = \frac{Sum(\theta_i^t\theta^{t-1})}{\sqrt{Sum((\theta_i^t)^2)Sum((\theta^{t-1})^2)}}. \tag{12}$$

We do not determine whether θ_i^t is poisonous based on the magnitude of Cos_i^t, but rather on whether Cos_i^t is the outlier. Poisonous updates deviate in the direction away from the majority of semi-honest users' updates, indicating that their cosine similarities would also be different from the semi-honest in magnitude. We construct the defense strategy based on the above. The details of the strategy are shown in Algorithm 4.

Assuming there are k users passing the verification, the CS sorts the cosine similarities of whom in descending order. The result is denoted as $r = \{cos_1, cos_2, \ldots, cos_k\}$. The CS employs the median in r, denoted as cos_m, as the benign reference benchmark. If $cos_i/cos_m > \delta$ or $cos_i/cos_m < \frac{1}{\delta}$, where δ is a threshold, then θ_i^t will be considered as a poisonous update and discarded. Similarly, the CS sorts the magnitudes of users' updates in descending order, denoted as $s = \{sum_1, sum_2, \ldots, sum_k\}$. The median is denoted as sum_m. If $sum_i/sum_m > \varepsilon$ or $sum_i/sum_m < \frac{1}{\varepsilon}$, where ε is a threshold, then θ_i^t will be considered poisonous and discarded.

After undergoing filtering based on magnitude and direction, the remaining updates are aggregated. Assuming that there are j updates retained, the CS performs the aggregation as follows:

$$[\![\theta^t]\!] = [\![\theta_1^t]\!] + [\![\theta_2^t]\!] + \cdots + [\![\theta_j^t]\!]. \tag{13}$$

Algorithm 3: Verification.

Input: The out of Algorithm 2 and $[\![\theta^{t-1}]\!]$.
Output: k.

CS:
Initialize $k = 0$;
for $i \in [1, n]$ **do**
  $[\![(\theta_i^t)^2]\!] \leftarrow He.Mul([\![\theta_i^t]\!], [\![\theta_i^t]\!])$;
  $[\![(\theta_i^t)^2 + \alpha - \beta]\!]_s \leftarrow He.Add([\![(\theta_i^t)^2]\!], [\![\alpha]\!], -[\![\beta]\!])$;
  $[\![\theta_i^t\theta^{t-1}]\!] \leftarrow He.Mul([\![\theta_i^t]\!], [\![\theta^{t-1}]\!])$;
  $[\![\theta_i^t\theta^{t-1} + \alpha - \beta]\!]_s \leftarrow He.Add([\![\theta_i^t\theta^{t-1}]\!], [\![\alpha]\!], -[\![\beta]\!])$
  $[\![(\theta_i^t)^2 + \alpha - \beta]\!]_s \cdot sk_1 \leftarrow PD([\![(\theta_i^t)^2 + \alpha - \beta]\!]_s, sk_1)$;
  $[\![\theta_i^t\theta^{t-1} + \alpha - \beta]\!]_s \cdot sk_1 \leftarrow PD([\![\theta_i^t\theta^{t-1} + \alpha - \beta]\!]_s, sk_1)$;
  $(\theta_i^t)^2 + \alpha - \beta \leftarrow CD([\![(\theta_i^t)^2 + \alpha - \beta]\!]_u \cdot sk_2, [\![(\theta_i^t)^2 + \alpha - \beta]\!]_s \cdot sk_1)$;
  $\theta_i^t\theta^{t-1} + \alpha - \beta \leftarrow CD([\![\theta_i^t\theta^{t-1} + \alpha - \beta]\!]_u \cdot sk_2, [\![\theta_i^t\theta^{t-1} + \alpha - \beta]\!]_s \cdot sk_1)$;
  $[\![\theta_i^t\theta^{t-1} + (\theta_i^t)^2]\!]_s \leftarrow He.Add([\![(\theta_i^t)^2]\!], [\![\theta_i^t\theta^{t-1}]\!])$
  $[\![\theta_i^t\theta^{t-1} + (\theta_i^t)^2]\!]_s \cdot sk_1 \leftarrow PD([\![\theta_i^t\theta^{t-1} + (\theta_i^t)^2]\!]_s, sk_1)$
  $\theta_i^t\theta^{t-1} + (\theta_i^t)^2 \leftarrow CD([\![\theta_i^t\theta^{t-1} + (\theta_i^t)^2]\!]_s \cdot sk_1, [\![\theta_i^t\theta^{t-1} + (\theta_i^t)^2]\!]_u \cdot sk_2)$
  Compute $Sum(\theta_i^t\theta^{t-1} + (\theta_i^t)^2)_s$
  if $Sum((\theta_i^t)^2 + \alpha - \beta) + Sum(\theta_i^t\theta^{t-1} + \alpha - \beta) = Sum(\theta_i^t\theta^{t-1} + (\theta_i^t)^2)$ **then**
    k++;
  else
    Discard θ_i^t;
  end if
end for

Then the CS computes $[\![\theta^t]\!] \cdot sk_1$ and broadcasts it with $[\![\theta^t]\!]$ and j to all users. Users decrypt $[\![\theta^t]\!]$ and update ω^{t-1} as follows:

$$\theta_t \leftarrow CD([\![\theta^t]\!] \cdot sk_1, [\![\theta^t]\!] \cdot sk_2), \omega^t = \omega^{t-1} + \frac{\theta^t}{j}. \tag{14}$$

VI. Analysis of the PurifyFL Protocol

In this section, we conduct a comprehensive analysis of PurifyFL, including security analysis, comparisons and efficiency analysis.

A. Security Analysis

In PurifyFL, the semi-honest server can not compromise the confidentiality of the updates. Poisoners also cannot learn any private information about other users' updates. We give strict security proof of PurifyFL, which is constructed by the standard hybrid argument [12].

Theorem 1. (The confidentiality of local updates and aggregated update against semi-honest server): Given a user set $U = \{U_1, U_2, \ldots, U_m\}$, a semi-honest server C, there exists a probabilistic polynomial-time (PPT) simulator SIM, such that the output of SIM is computationally indistinguishable from

Authorized licensed use limited to: National University of Modern Languages. Downloaded on July 31,2025 at 07:01:33 UTC from IEEE Xplore. Restrictions apply.

Algorithm 4: Defense.

Input: $[\![\theta_i^t]\!], i \in [1, k]$.
Output: $[\![\theta_t]\!] \cdot sk_1$ and j.
CS:
Initialize $j = 0$;
Calculate the median cos_m;
Calculate the median sum_m;
for $i \in [1, k]$ **do**
  if $cos_i/cos_m > \delta$ or $cos_i/cos_m < \frac{1}{\delta}$ or $sum_i/sum_m > \varepsilon$ or $sum_i/sum_m < \frac{1}{\varepsilon}$ **then**
    Discard θ_i^t;
  end if
  $[\![\theta_t]\!] \leftarrow [\![\theta_t]\!] + [\![\theta_i^t]\!]$, j++;
end for
$[\![\theta_t]\!] \cdot sk_1 \leftarrow PD([\![\theta_t]\!], sk_1)$;
Return $[\![\theta_t]\!] \cdot sk_1$ and j to all users;

$\mathrm{REAL_C}^U$, which represents the view of C during the execution of the protocol in the real world:

$$\mathbf{REAL_C}^{\boldsymbol{U}} \equiv \mathbf{SIM_C}^{\boldsymbol{U}},$$

where "≡" denotes computational indistinguishability.

Proof: $\mathrm{REAL_C}^U$ contains all the information that central server C can receive. The construction of the PPT simulator is based on a sequence of subsequent modifications to the random variables in $\mathrm{REAL_C}^U$.

Hyb_0. A random variable is initialized and its distribution of is indistinguishable from $\mathrm{REAL_C}^U$.

Hyb_1. In this hybrid, the behavior of the simulated users $U_i \in U$ is adapted. For U_i, it no longer encrypts the original update vector θ_i^t, but a random vector V_i^t with pk. Since only the contents of the ciphertexts change, the semantic security of the CKKS cryptosystem [27] ensures the indistinguishability between Hyb_1 and Hyb_0.

Hyb_2. In this hybrid, the simulated user generates random noise ζ and η to replace α and β. Then it calculates $[\![(V_i^t)^2 + \zeta - \eta]\!]_u \cdot sk_2$, $[\![V_i^t V^{t-1} + \zeta - \eta]\!]_u \cdot sk_2$ and $[\![V_i^t V^{t-1} + (V_i^t)^2]\!]_u \cdot sk_2$. Since only the contents of the ciphertexts change, the semantic security of HE and ensure the indistinguishability between Hyb_2 and Hyb_1.

Hyb_3. In this hybrid, the server calculates $[\![(V_i^t)^2 + \zeta - \eta]\!]_s \cdot sk_1$, $[\![V_i^t V^{t-1} + \zeta - \eta]\!]_s \cdot sk_1$ and $[\![V_i^t V^{t-1} + (V_i^t)^2]\!]_s \cdot sk_1$. Since $[\![(V_i^t)^2 + \zeta - \eta]\!]_s \cdot sk_1$, $[\![V_i^t V^{t-1} + \zeta - \eta]\!]_s \cdot sk_1$ and $[\![V_i^t V^{t-1} + (V_i^t)^2]\!]_s \cdot sk_1$ are the outputs of homomorphic operation and only the contents of the ciphertexts change, the semantic security of HE ensures the indistinguishability between Hyb_3 and Hyb_2.

Hyb_4. The server decrypts ciphertexts to achieve $(V_i^t)^2 + \zeta - \eta$, $V_i^t V^{t-1} + \zeta - \eta$ and $V_i^t V^{t-1} + (V_i^t)^2$. Since ζ and η are random, $(V_i^t)^2 + \zeta - \eta$ and $V_i^t V^{t-1} + \zeta - \eta$ are also random. The server does not have knowledge about V^{t-1}, it can not achieve V_i^t by $V_i^t V^{t-1} + (V_i^t)^2$. Therefore Hyb_4 and Hyb_3 are indistinguishable.

Hyb_5. In this hybrid, the input of the aggregation process is substituted from $[\![\theta_i^t]\!]$ to $[\![V_i^t]\!]$. And the output of the aggregation is $\prod_{i=1}^{i=k} [\![V_i]\!]$, which is the output of additive homomorphism. The semantic security of HE ensures the indistinguishability between Hyb_5 and Hyb_4.

As mentioned above, based on the semantic security of HE, the outputs of SIM and REAL are proven indistinguishable. Therefore, in PurifyFL, the honest and curious central server CS does not compromise the user's privacy.

Theorem 2. (The confidentiality of local updates against semi-honest users, poisoners, and eavesdroppers): Given a user set $U = \{U_1, U_2, \ldots, U_m\}$, an adversary U_*, there exists a PPT simulator SIM, such that the output of SIM is computationally indistinguishable from $\mathrm{REAL_{U_*}}^U$, which represents the view of U_* during the execution of the protocol in the real world:

$$\mathbf{REAL_{U_*}}^{\boldsymbol{U}} \equiv \mathbf{SIM_{U_*}}^{\boldsymbol{U}},$$

where "≡" denotes computational indistinguishability.

Proof: As mentioned in Section III, adversaries are capable of eavesdropping on any messages received by the server. We prove that adversaries cannot get other users' local updates.

Hyb_0. A random variable is initialized, the distribution of which is indistinguishable from $\mathrm{REAL_C}^U$.

Hyb_1. In this hybrid, the behavior of the simulated users $U_i \in U$ is adapted. For U_i, it no longer encrypts the original update vector θ_i^t, but a random vector V_i^t with pk. Since only the contents of the ciphertexts change, the semantic security of HE ensures the indistinguishability between Hyb_1 and Hyb_0.

Hyb_2. In this hybrid, the simulated user generates random noise ζ and η to replace α and β. Then it calculates $[\![(V_i^t)^2 + \zeta - \eta]\!]_u \cdot sk_2$, $[\![V_i^t V^{t-1} + \zeta - \eta]\!]_u \cdot sk_2$ and $[\![V_i^t V^{t-1} + (V_i^t)^2]\!]_u \cdot sk_2$. Other users can not decrypt ciphertexts by sk_2 and eavesdroppers also can not decrypt ciphertexts, so Hyb_2 and Hyb_1 are indistinguishable.

Hyb_3. In this hybrid, the server calculates $[\![V^t]\!] \cdot sk_1$. Since eavesdroppers do not hold sk_1, Hyb_3 and Hyb_2 are indistinguishable.

As mentioned above, based on the semantic security of HE, the outputs of SIM and REAL are indistinguishable. Therefore, in PurifyFL, adversaries cannot get the local updates of users.

B. Comparisons

We conduct a comparison of PurifyFL with five typical protocols, PEFL [23], ShieldFL [24], Krum [19], FL-Trust [21] and DetectPMFL [25], which is shown in Table I.

Both Krum and FL-Trust employ the standard FL architecture with only one server and no additional interactions. However, both of them only focus on poisoning attacks and ignore protecting the updates. Krum relies solely on Euclidean distance to defend against poisoning attacks. Poisoners can launch poisoning attacks in both directions and magnitudes. If a protocol can only detect one of these aspects, it means that the protocol is not robust for poisoning attacks. Euclidean distance reflects the distance between two points in high-dimensional space and it does not reflect the direction and magnitude of the updates, that's why we think Krum is not robust for poisoning attacks. FL-Trust limits the magnitude of updates by normalizing them and detects the

Authorized licensed use limited to: National University of Modern Languages. Downloaded on July 31,2025 at 07:01:33 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISONS OF PROTOCOLS DEFENDING AGAINST POISONING ATTACKS

Protocol	Single server	Non-interactive	Privacy of local updates	Privacy of aggregated update	Robustness of detection mechanism
Krum [19]	√	√	×	×	×
FL-Trust [21]	√	√	×	×	√
PEFL [23]	×	√	×	×	×
ShieldFL [24]	×	√	√	√	√
DetectPMFL [25]	√	×	×	√	×
PurifyFL	√	√	√	√	√

TABLE II
COMPUTATION AND COMMUNICATION OVERHEAD COMPARISON

Phase	ShieldFL		PurifyFL	
	Computation	Communication	Computation	Communication
System initialization	$nm(2C_e + C_m)$	$O(nm)$	$2nm(C_m + C_a)/N$	$O(4nm/N)$
Similarity calculation	$nm(15C_e + 15C_m + C_{Enc})$	$O(10nm)$	$nm(6C_{Enc} + 20C_m + 22C_a)/N$	$O(12nm/N)$
Secure aggregation	$nm(3C_e + 2C_m)$	$O(2nm)$	$2nm(2C_a + C_m)/N$	$O(4nm/N)$
Total	$nm(20C_e + 18C_m + C_{Enc})$	$O(12nm)$	$nm(24C_m + 6C_{Enc} + 24C_a)/N$	$O(20nm/N)$

direction of updates with cosine similarity. Compared to Krum, the detection mechanism of FL-Trust is more robust.

The construction of PEFL relies on two servers. In PEFL, two servers execute protocols interactively and there is no additional interaction between the users and the servers. Although PEFL is constructed based on homomorphic encryption, it still suffers from privacy leakage [33]. PEFL decides whether an update is poisonous based only on its Pearson correlation coefficient and does not check the magnitude of the update, thus PEFL is also not robust for poisoning attacks.

ShieldFL is similar to PEFL in that it is also dual-server-based and non-interactive. ShieldFL constructs a series of secure computational protocols by two-trapdoor homomorphic encryption and it can protect both the local updates and the global update. The detection mechanism of ShieldFL is similar to FL-trust's, which is also robust.

DetectPMFL is a single-server-based protocol, however, in each epoch, the server sends the encrypted intermediate results to users for decryption, which incurs additional interactions. Therefore, DetectPMFL does not achieve non-interactivity. The private key that can decrypt the ciphertexts of updates is held by all users, which means that the server cannot obtain users' local updates and the aggregated update, but a user can decrypt other users' updates. Therefore, DetectPMFL can only ensure the privacy of the aggregated update. Since DetectPMFL only checks the direction while ignoring the magnitude, it is not robust for poisoning attacks.

PurifyFL overcomes the drawbacks of the aforementioned protocols simultaneously. As a non-interactive protocol based on a single server, PurifyFL ensures the privacy of users' local updates and aggregated update. Moreover, PurifyFL checks the magnitude and direction of updates at the same time. The experiments in Section VI prove that PurifyFL is robust for poisoning attacks.

C. Efficiency Analysis

We analyze the communication and computational overheads of PurifyFL. We also compare PurifyFL with ShieldFL. The comparison between ShieldFL and PurifyFL is presented in Table II, where C_e, C_m, C_a and C_{Enc} represent the modular exponentiation, multiplication, addition operation and encryption, respectively. In Table II, we only count the overheads that depend on both m and n, which represent the length of the updates and the number of users, respectively. The other operations such as decrypting the magnitude of updates only depended on n, which is negligible.

The computation overhead in the system initialization phase is mainly the decryption of initial model parameters, which costs $2\,\mathrm{nm}(C_m + C_a)/N$. N is the polynomial modulus degree of CKKS. In this paper, the value of N is 8192, which means that the user can encrypt 4096 messages at a time. The communication overhead of this phase is $O(4\,\mathrm{nm}/N)$, which comes from the transmission of $[\![\omega^0]\!]$ and $[\![\omega^0]\!] \cdot sk_1$.

In the similarity computation phase, the encryption of θ_i^t, α and β costs $6\,\mathrm{nm}(C_{Enc})/N$. The computation of $[\![(\theta_i^t)^2 + \alpha - \beta]\!]_s$ and $[\![(\theta_i^t)^2 + \alpha - \beta]\!]_u$ costs $2\,\mathrm{nm}(2C_m + C_a)/N$. The computation of $[\![\theta_i^t\theta^{t-1} + \alpha - \beta]\!]_u$ and $[\![\theta_i^t\theta^{t-1} + \alpha - \beta]\!]_s$ costs $2\,\mathrm{nm}(2C_m + 2C_a)/N$. The total computational cost of this phase is $nm(6C_{Enc} + 20C_m + 22C_a)/N$. The communication overhead is $O(12\,\mathrm{nm})/N$, consisting of the transmission of $[\![\alpha]\!]$, $[\![\beta]\!]$, $[\![\theta_i^t]\!]$, $[\![(\theta_i^t)^2 + \alpha - \beta]\!]_u \cdot sk_2$, $[\![\theta_i^t\theta^{t-1} + \alpha - \beta]\!]_u \cdot sk_2$ and $[\![\theta_i^t\theta^{t-1} + (\theta_i^t)^2]\!]_u \cdot sk_2$.

The total computation cost of the secure aggregation phase is $2\,\mathrm{nm}(2C_a + C_m)/N$, consisting of aggregating users' updates on ciphertexts and users' decrypting the global update, the overheads of which are $2\,\mathrm{nm}(C_m)/N$ and $2\,\mathrm{nm}(C_m + C_a)/N$, respectively. The communication cost is $O(4\,\mathrm{nm}/N)$, including the transmission of the encrypted global update and the result of pre-decryption from the server to users.

As shown in Table II, the computational and communication overhead of PurifyFL is less than that of ShieldFL, which is also true in the experiment.

VII. PERFORMANCE EVALUATION

In this section, we evaluate the performance of PurifyFL. We first demonstrate the setup of experiments, then we compare PurifyFL with existing works on two datasets.

Authorized licensed use limited to: National University of Modern Languages. Downloaded on July 31,2025 at 07:01:33 UTC from IEEE Xplore. Restrictions apply.

A. Experimental Setup

We simulate the whole process of federated learning on a PC with Intel i5-12600kf, 48 GB RAM and NVIDIA RTX 3080.

1) Datasets: We test PurifyFL with classification tasks on MNIST-10 and CIFAR-10. The degree of non-IID is the hyper-parameter α in the Dirichlet distribution. In this paper, we set α to 0.5.

2) Poisoning attacks: We employ four types of poisoning attacks to assess the effectiveness of the proposed PurifyFL. They are label flipping attacks [29], distributed backdoor attacks (DBA) [34], Gaussian noise attacks and label shuffle attacks [24].

Label flipping attacks and distributed backdoor attacks are targeted. We launch label flipping attacks by modifying the labels of a specific class, while the implementation of DBA is achieved by injecting the distributed backdoor in the training data. We compute the overall accuracy (OA) and attack success rate (SR, the proportion of the model outputs the labels predetermined by the attacker) to evaluate the proposed PurifyFL under targeted attacks. Additionally, for label flipping attacks, we also compute the source accuracy (SA, the accuracy of the class with modified labels).

Gaussian noise attacks and label shuffle attacks are untargeted, aiming to decrease the overall accuracy of the model. We simulate the above two attacks by introducing Gaussian noise to the training images and randomly shuffling labels, respectively. We compute the overall accuracy of the model to evaluate the proposed PurifyFL under untargeted attacks.

3) Model architectures and training settings: We use a convolutional neural network (CNN) model with 2 convolutional layers and 1 linear layer on MNIST-10, which is sufficient for the task of accurate classification. The images in CIFAR-10 have more complex features compared to MNIST-10, so we use a model with six convolutional layers and three linear layers to ensure accuracy. In addition, we take adaptive moment estimation (Adam) [35] as the optimizer. The learning rate we adopt is 0.0001. In the IID setting, δ and ε are set to 1.3, while in the non-IID setting, δ and ε are 1.5. The batch size is 64. The total number of users is 21, and the proportion of malicious users is 50%, which is the upper limit most of the existing works can tolerate.

B. Experimental Results

The performance of PurifyFL is evaluated in this subsection. We compare PurifyFL to previous works: Trimmed-mean [18], Krum [19], Bulyan [20], PEFL [23] and ShieldFL [24] are chosen to be compared with PurifyFL. In addition, we introduce FedAvg [12] as the baseline, which averages all updates, to show the devastation of the poisoning attacks. $None$ represents the unpoisoned baseline.

1) Evaluation on Targeted Attacks: The results is shown in Tables III and IV.

In the IID setting, for label flipping attacks, the SA of baseline on two datasets is only 28.1% and 20.0%, respectively. Meanwhile, the SA of None is 98.3% and 94.6%, respectively, which means that the simulated label flipping attacks disrupt the baseline successfully. On MNIST-10, PurifyFL reduces SR from 46.4% to 0.0% and increases SA from 28.1% to 97.5%. Similar significant improvements are also observed on CIFAR-10. In the Non-IID setting, compared to existing works, PurifyFL exhibits the lowest SR and the highest SA, remaining effective against label flipping attacks. In contrast, Krum is effective only in the IID setting, while Bulyan performs poorly in both settings due to the proportion of malicious users exceeding the scheme's tolerance limit.

Distributed backdoor attacks exhibit high SA in both settings. In the IID setting, all works except Bulyan mitigate distributed backdoor attacks. However, in the Non-IID setting, Trimmed-Mean and Krum fail to defend against it. Although ShieldFL and PEFL are effective in mitigating distributed backdoor attacks, they merely assign low weights to poisonous updates, which still participate in the aggregation. Consequently, the SA of them is higher than that of PurifyFL.

We notice that the OA of PurifyFL is slightly lower than that of PEFL and ShieldFL. We attribute this discrepancy to the nature

TABLE III
COMPARISONS ON TARGETED ATTACKS, MNIST-10

		DBA		Label flipping		
		SR	OA	SR	SA	OA
IID	Baseline [12]	93.3	91.5	46.4	28.1	86.1
	Trimmed [18]	8.7	91.2	0.9	96.3	90.4
	Krum [19]	6.1	87.2	2.8	90.8	88.7
	Bulyan [20]	83.6	87.8	73.3	20.2	86.6
	PEFL [23]	4.2	92.8	0.2	96.7	92.0
	ShieldFL [24]	6.5	92.5	0.6	94.5	92.2
	PurifyFL	**0.0**	92.3	**0.0**	**97.5**	91.1
	None	/	93.5	/	98.3	93.5
Non-IID	Baseline [12]	97.9	86.4	48.6	21.3	82.3
	Trimmed [18]	13.0	84.4	2.1	89.7	86.5
	Krum [19]	18.4	78.8	27.6	49.3	81.9
	Bulyan [20]	71.2	84.3	64.1	18.6	76.8
	PEFL [23]	9.8	87.9	4.2	88.4	86.7
	ShieldFL [24]	7.7	87.3	5.0	86.2	87.3
	PurifyFL	**4.3**	87.0	**2.2**	**89.9**	86.9
	None	/	88.9	/	94.6	88.9

OA refers to the overall accuracy of the model and SR refers to the attack success rate. SA is a specific metric in label flipping attacks, representing the source class accuracy, i.e., the accuracy of the class under attack.

TABLE IV
COMPARISONS ON TARGETED ATTACKS, CIFAR-10

		DBA		Label flipping		
		SR	OA	SR	SA	OA
IID	Baseline [12]	82.1	77.8	44.5	20.0	75.0
	Trimmed [18]	2.7	77.2	1.3	88.7	78.6
	Krum [19]	8.1	76.1	4.6	85.8	76.7
	Bulyan [20]	60.8	75.5	70.2	17.3	74.2
	PEFL [23]	2.3	80.2	6.8	81.5	81.0
	ShieldFL [24]	1.5	80.7	4.9	84.6	81.1
	PurifyFL	**0.0**	79.8	**0.8**	**89.7**	79.5
	None	/	81.3	/	90.2	81.3
Non-IID	Baseline[12]	82.4	67.2	70.3	12.5	68.7
	Trimmed [18]	48.5	70.3	8.9	75.1	70.4
	Krum [19]	77.9	70.7	46.8	39.6	72.2
	Bulyan [20]	84.9	68.6	78.2	6.5	69.4
	PEFL [23]	12.7	73.6	7.6	77.2	72.8
	ShieldFL [24]	7.3	73.8	9.9	75.0	74.5
	PurifyFL	**3.4**	72.4	**2.7**	**82.4**	73.3
	None	/	75.3	/	88.0	75.3

Authorized licensed use limited to: National University of Modern Languages. Downloaded on July 31,2025 at 07:01:33 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPARISONS ON UNTARGETED ATTACKS, MNIST-10

Setting	Attack	Baseline [12]	Trimmed [18]	Krum [19]	Bulyan [20]	PEFL [23]	ShieldFL [24]	PurifyFL	None
IID	Gaussian noise	85.3	88.2	87.2	85.9	89.3	88.5	**90.6**	93.5
	Label shuffle	89.0	90.6	89.7	87.4	89.6	89.8	**91.0**	
Non-IID	Gaussian noise	77.3	83.6	70.3	72.6	84.4	87.0	**87.3**	88.9
	Label shuffle	80.3	82.6	74.1	69.8	83.7	86.2	**87.5**	

The aim of the untargeted attacks is to decrease the overall accuracy of the model, so only the overall accuracy is counted in the table.

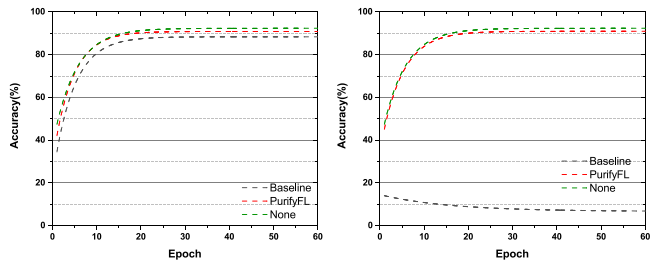
TABLE VI
COMPARISONS ON UNTARGETED ATTACKS, CIFAR-10

Setting	Attack	Baseline [12]	Trimmed [18]	Krum [19]	Bulyan [20]	PEFL [23]	ShieldFL [24]	PurifyFL	None
IID	Gaussian noise	37.0	74.8	41.8	35.1	76.9	58.8	**80.1**	81.3
	Label shuffle	45.7	71.1	48.2	32.4	73.4	62.5	**79.4**	
Non-IID	Gaussian noise	31.0	66.6	28.3	26.0	60.2	64.2	**73.5**	75.3
	Label shuffle	33.8	63.0	34.5	30.8	57.2	66.9	**72.2**	

of the targeted attacks, which focus solely on specific targets, while malicious updates are still beneficial for improving the accuracy of other classes. PurifyFL directly removes malicious updates, meaning that with the proportion of malicious users being 50%, only half of the training data is used for training the model. Considering the devastation of poisoning attacks, we consider it acceptable to trade a slight decrease in OA for a more effective defense against poisoning attacks.

2) Evaluation on Untargeted Attacks: The performance of PurfifyFL under untargeted attacks is presented in Tables V and VI. PurifyFL is effective in improving the accuracy of the model under two data distributions, with only a slight decrease in accuracy compared to the baseline. Compared to the baseline, there is a certain degree of improvement in the accuracy of Trimmed-mean. However, compared to the unpoisoned baseline, the accuracy of Trimmed-mean is not satisfactory. Both Krum and Bulyan fail to defend against untargeted attacks. Krum is only effective on MNIST-10 with IID settings. With the proportion of malicious users exceeding the designed limit, Bulyan's performance is even worse than the baseline. The malicious updates of targeted attacks still contribute to the overall accuracy. Unlike targeted attacks, updates of untargeted attacks only diminish the overall accuracy of the model. PEFL and ShieldFL do not directly remove malicious updates, but instead assign low weights to them. Their strategy makes them perform better in terms of overall accuracy against targeted attacks. However, it also leads to a degradation of model performance against untargeted attacks.

We observe that untargeted attacks on MNIST-10 do not have a significant impact on the baseline. The maximum accuracy difference between the baseline and the unpoisoned baseline is only about 11%. In fact, attackers can amplify model updates to enhance untargeted attacks. Targeted attacks are restricted from affecting the accuracy of other classes, whereas untargeted attacks do not have such a restriction, so malicious users can amplify their updates without limit. As shown in Fig. 4, for the same proportion of poisoners, the malicious updates in Fig. 4(b) are amplified by a factor of 10, and the accuracy of the baseline drops from 89.6% to 5.3% compared to that in Fig. 4(a), while the



(a) 30% of malicious users. (b) 30% of malicious users, amplified updates

Fig. 4. The effect of amplified poisonous updates, untargeted attacks, MNIST-10. In (a), the magnitudes of poisonous updates maintain normal, while in (b), they are amplified tenfold. With the same proportion of poisoners, the amplified poisoning attacks show significant effects.

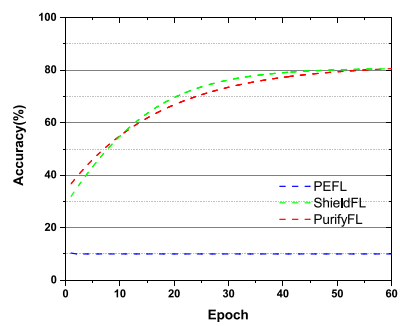
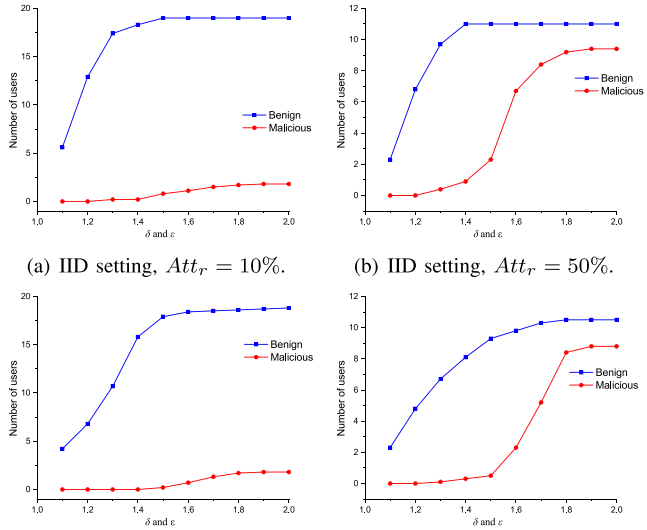


Fig. 5. Untargeted attacks, 50% of malicious users, CIFAR-10.

accuracy of PurifyFL is hardly affected. In fact, due to the lack of the amplitude detection mechanism, PEFL is highly vulnerable when confronted with untargeted attacks. As shown in Fig. 5, we amplify the poisonous updates by a factor of 10, which leads to the gradient explosion in PEFL after merely two epochs, but PurifyFL and ShieldFL remain effective in defending against poisoning attacks.

In summary, the performance of PurifyFL is satisfactory, which can significantly mitigate poisoning attacks and has advantages in accuracy and robustness over existing protocols against poisoning attacks in PPFL.

Authorized licensed use limited to: National University of Modern Languages. Downloaded on July 31,2025 at 07:01:33 UTC from IEEE Xplore. Restrictions apply.



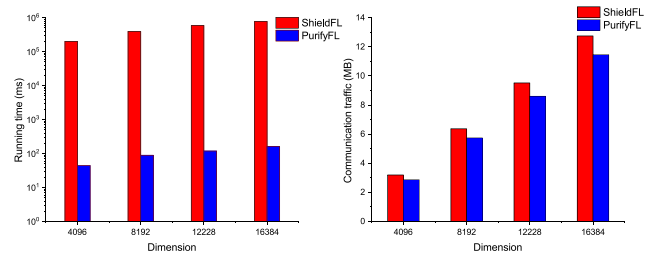
(a) IID setting, $Att_r = 10\%$. (b) IID setting, $Att_r = 50\%$.

(c) Non-IID setting, $Att_r = 10\%$. (d) Non-IID setting, $Att_r = 50\%$.

Fig. 6. The influence of hyperparameters. Att_r refers to the proportion of poisoners.

3) Hyperparameters: We take the label-flipping attack as an example, conducting tests to evaluate the impact of hyperparameters δ and ε on the final number of clients participating in aggregation. Att_r represents the proportion of malicious users. For simplicity, we set $\delta = \varepsilon$. As shown in Fig. 6, when the hyperparameters are small, although few malicious users participate in the aggregation, most benign users are also excluded. Conversely, too large hyperparameters result in almost all malicious users participating in the aggregation. From the experimental results, it is observed that the optimal hyperparameters vary based on the proportion of malicious users. Since knowing the proportion in advance is not practical, the hyperparameters are determined based on $Att_r = 50\%$. Under the IID setting, we set the hyperparameters to 1.3. With $Attr = 50\%$, the number of benign users and malicious users is 9.7 and 0.4, respectively, and 96% of malicious users are excluded while only 11% of benign users are lost. With $Attr = 10\%$, the number of benign users and malicious users is 17.4 and 0.2, respectively, with the former being 87 times the latter. Under the Non-IID setting, due to significant differences in users' model updates, we set the hyperparameters to 1.5 to balance aggregating more updates from benign users and excluding malicious users. After calculations, with $Att_r = 10\%$ and $Att_r = 50\%$, the number of benign users is 90 times and 18 times the number of malicious users, respectively. Furthermore, due to the restricted magnitude of updates from malicious users, meaning that malicious users are unlikely to have a negative impact on the model.

4) Efficiency Performance: We test the running time (including only the time spent on encryption, excluding model training) and communication traffic generated during the execution of PurifyFL and compare PurifyFL with ShieldFL. ShieldFL adopts the two-trapdoor HE protocol with the 80-bit security level, which has been proven not secure. In this paper, we adjust the security level of ShieldFL to 112-bit and the security level of PurifyFL is 128-bit. As shown in Fig. 7, the running



(a) Computational overhead. (b) Communication overhead.

Fig. 7. The computational and communication overheads of per user. Dimension refers to the length of the model updates.

time and communication overhead of PurifyFL and ShieldFL exhibit a linear increase with the dimension of model updates. When the dimension of model updates is 4096, the runtime and communication overhead for PurifyFL are 44 ms and 2.87 MB, respectively, while for ShieldFL, they are 200,115 ms and 3.19 MB, respectively. The running efficiency of PurifyFL is much higher than that of ShieldFL, which aligns with Table II. In terms of communication overheads, because a ciphertext of the CKKS cryptosystem is much larger than that of the two-trapdoor HE cryptosystem, PurifyFL only has a slight advantage over ShieldFL.

VIII. Conclusion

In this paper, we propose PurifyFL, an FL protocol that can preserve privacy and mitigate poisoning attacks. PurifyFL is a non-interactive protocol and only relies on a single server, whereas no existing works can achieve both of these aspects simultaneously. Experimental results indicate that PurifyFL can mitigate poisoning attacks significantly. In this paper, the server simply takes the average of users' updates as the global update, which incurs a significant degradation of the model performance when data is non-IID. A feasible solution to this problem is personalized federated learning. In future work, we will introduce personalized aggregation techniques to generate models tailored to users while defending against poisoning attacks.

References

[1] D. D. Lopes et al., "Machine learning partners in criminal networks," *Sci. Rep.*, vol. 12, no. 1, 2022, Art. no. 15746.
[2] H. V. Ribeiro et al., "Deep learning criminal networks," *Chaos, Solitons Fractals*, vol. 172, 2023, Art. no. 113579.
[3] X. Wang, W. Fan, X. Hu, J. He, and C.-H. Chi, "Differential privacy-preserving of multi-party collaboration under federated learning in data center networks," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 8, no. 2, pp. 1223–1237, Apr. 2024.
[4] L. Li, Y. Fan, M. Tse, and K.-Y. Lin, "A review of applications in federated learning," *Comput. Ind. Eng.*, vol. 149, 2020, Art. no. 106854.
[5] V. Mothukuri, R. M. Parizi, S. Pouriyeh, Y. Huang, A. Dehghantanha, and G. Srivastava, "A survey on security and privacy of federated learning," *Future Gener. Comput. Syst.*, vol. 115, pp. 619–640, 2021.
[6] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 691–706.
[7] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 14774–14784.
[8] B. Zhao, K. R. Mopuri, and H. Bilen, "iDLG: Improved deep leakage from gradients," 2020, *arXiv:2001.02610*.

Authorized licensed use limited to: National University of Modern Languages. Downloaded on July 31,2025 at 07:01:33 UTC from IEEE Xplore. Restrictions apply.

[9] K. Wei et al., "Federated learning with differential privacy: Algorithms and performance analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 3454–3469, 2020.
[10] L. Yu, L. Liu, C. Pu, M. E. Gursoy, and S. Truex, "Differentially private model publishing for deep learning," in *Proc. IEEE Symp. Secur. Privacy*, IEEE, 2019, pp. 332–349.
[11] M. Sun and W. P. Tay, "On the relationship between inference and data privacy in decentralized IoT networks," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 852–866, 2020.
[12] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1175–1191.
[13] J. H. Bell, K. A. Bonawitz, A. Gascón, T. Lepoint, and M. Raykova, "Secure single-server aggregation with (poly) logarithmic overhead," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 1253–1269.
[14] S. Kadhe, N. Rajaraman, O. O. Koyluoglu, and K. Ramchandran, "FastSecAgg: Scalable secure aggregation for privacy-preserving federated learning," 2020, *arXiv:2009.11248*.
[15] L. T. Phong, Y. Aono, T. Hayashi, L. Wang, and S. Moriai, "Privacy-preserving deep learning via additively homomorphic encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 5, pp. 1333–1345, May 2018.
[16] S. Sav et al., "POSEIDON: Privacy-preserving federated neural network learning," in *Proc. 28th Annu. Netw. Distrib. Syst. Secur. Symp.*, 2021.
[17] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, "BatchCrypt: Efficient homomorphic encryption for cross-silo federated learning," in *Proc. USENIX Annu. Tech. Conference. USENIX Assoc.*, 2020, pp. 493–506.
[18] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5650–5659.
[19] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 118–128.
[20] R. Guerraoui et al., "The hidden vulnerability of distributed learning in Byzantium," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 3521–3530.
[21] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Proc. 28th Annu. Netw. Distrib. Syst. Secur. Symp.*, 2021.
[22] A. Yazdinejad, A. Dehghantanha, H. Karimipour, G. Srivastava, and R. M. Parizi, "A robust privacy-preserving federated learning model against model poisoning attacks," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 6693–6708, 2024.
[23] X. Liu, H. Li, G. Xu, Z. Chen, X. Huang, and R. Lu, "Privacy-enhanced federated learning against poisoning adversaries," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4574–4588, 2021.
[24] Z. Ma, J. Ma, Y. Miao, Y. Li, and R. H. Deng, "ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1639–1654, 2022.
[25] Z. Zhang, N. He, Q. Li, K. Wang, H. Gao, and T. Gao, "DetectPMFL: Privacy-preserving momentum federated learning considering unreliable industrial agents," *IEEE Trans. Ind. Informat.*, vol. 18, no. 11, pp. 7696–7706, Nov. 2022.
[26] J. Chen, H. Yan, Z. Liu, M. Zhang, H. Xiong, and S. Yu, "When federated learning meets privacy-preserving computation," *ACM Comput. Surv.*, vol. 56, no. 12, pp. 1–36, 2024.
[27] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Springer, 2017, pp. 409–437.
[28] S. Chen, X. Wang, P. Zhou, W. Wu, W. Lin, and Z. Wang, "Heterogeneous semi-asynchronous federated learning in Internet of Things: A multi-armed bandit approach," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 6, no. 5, pp. 1113–1124, Oct. 2022.
[29] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," in *Proc. 29th Int. Conf. Mach. Learn.*, 2012, pp. 1467–1474.
[30] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2020, pp. 2938–2948.
[31] Z. Wang, Q. Kang, X. Zhang, and Q. Hu, "Defense strategies toward model poisoning attacks in federated learning: A survey," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2022, pp. 548–553.
[32] J. Wang, G. Xu, W. Lei, L. Gong, X. Zheng, and S. Liu, "CPFL: An effective secure cognitive personalized federated learning mechanism for industry 4.0," *IEEE Trans. Ind. Informat.*, vol. 18, no. 10, pp. 7186–7195, Oct. 2022.
[33] T. Schneider, A. Suresh, and H. Yalame, "Comments on "privacy-enhanced federated learning against poisoning adversaries"," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 1407–1409, 2023.
[34] C. Xie, K. Huang, P. Y. Chen, and B. Li, "DBA: Distributed backdoor attacks against federated learning," in *Proc. 8th Int. Conf. Learn. Representations*, 2020.
[35] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.

Yanli Ren received the M.S. degree in applied mathematics from Shaanxi Normal University, Xi'an, China, in 2005, and the Ph.D. degree in computer science and technology from Shanghai Jiao Tong University, Shanghai, China, in 2009. She is currently a Professor with the School of Communication and Information Engineering, Shanghai University, Shanghai. She has authored or coauthored more than 80 quality papers, including publications in IEEE Transactions on Dependable and Secure Computing, IEEE Transactions on Services Computing, IEEE Transactions on Industrial Informatics, IEEE Transactions on Cloud Computing, IEEE Transactions on Emerging Topics in Computational Intelligence and AsiaCCS. Her research interests include applied cryptography, secure outsourcing computation, blockchain security, AI security, and network security.

Zhe Yang is currently working toward the M.S degree with the School of Communication and Information Engineering, Shanghai University, Shanghai, China. His research interests include applied cryptography, secure outsourcing computation, and privacy-preserving machine learning.

Guorui Feng received the B.S. and M.S. degrees in computational mathematic from Jilin University, Changchun, China, in 1998 and 2001, respectively, the Ph.D. degree in electronic engineering from Shanghai Jiaotong University, Shanghai, China, 2005. From January 2006 to December 2006, he was an Assistant Professor with East China Normal University, Shanghai. In 2007, he was a Research Fellow with Nanyang Technological University, Singapore. He is currently with the School of Communication and Information Engineering, Shanghai University, Shanghai. His research interests include image processing, image analysis and computational intelligence.

Xinpeng Zhang received the B.S. degree in computational mathematics from Jilin University, Changchun, China, in 1995, and the M.S. and Ph.D. degrees in communication and information system from Shanghai University, Shanghai, China, in 2001 and 2004, respectively. Since 2004, he has been with the Faculty of the School of Communication and Information Engineering, Shanghai University, where he is currently a Professor. From 2010 to 2011, he was with the State University of New York at Binghamton, Binghamton, NY, USA, as a Visiting Scholar, and Konstanz University, Konstanz, Germany, as an experienced Researcher sponsored by the Alexander von Humboldt Foundation from 2011 to 2012. He has authored or coauthored more than 200 papers in his research interests which include multimedia security, image processing, and digital forensics.

Authorized licensed use limited to: National University of Modern Languages. Downloaded on July 31,2025 at 07:01:33 UTC from IEEE Xplore. Restrictions apply.